1

foo (6)

Stack		Registers	
bar (7)	int y=0	return (2)	
bar (6)	int y=1	return y + bar (0)	(3)
bar (5)	int y=2	return y + bar (1)	(5)
bar (4)	int y=3	return y + bar (2)	(8)
bar (3)	int y=4	return y + bar (3)	(12)
bar (2)	int y=5	return y + bar (4)	(17)
foo (1)	int x=6	return x + bar (5)	
main	result	foo (6) (23)	return 0;

#2

Stack	Registers
char input [] = "101010"	
func 6	&input, place=5, value=0 mul 0 return 0
func 5	&input, place=4, value=1 mul=2^1, return 2^1 + func(input, 5, 0)
func 4	&input, place=3, value=2 mul=0, return 0 + func(input, 4, 1)
func 3	&input place=2, value=3 mul=2^3, return 2^3 + func(input, 3, 2)
func 2	&input place=1, value=4 mul=0, return 0 + func(input, 2, 3) 10
func 1	&input place=0, value=5 mul=2^5, return 2^5 + func(input, 1, 4)
main	return func(&input, 0, 5) return 0;

42

#3

Stack	Registers

#4 - Memoization

→ Correct answer includes

1) Description of how storing intermediate values to improve computing performance

2) Will likely cite Fibonnacci

→ Makes <u>two</u> function calls on the stack each call

3) As the calls increase, eventually the stack will overflow

4) Allocate a static array on the stack Use of intermediate values significantly reduces calls on the stack